

```
In [2]: !pip install opencv-python
```

```
Collecting opencv-python
  Downloading opencv_python-5.0.0.93-cp37-abi3-manylinux_2_28_x86_64.whl.metadata (19 kB)
Collecting numpy>=2 (from opencv-python)
  Downloading numpy-2.5.3-cp314-cp314-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (6.6 kB)
Downloading opencv_python-5.0.0.93-cp37-abi3-manylinux_2_28_x86_64.whl (73.8 MB)
_____ 73.8/73.8 MB 38.0 MB/s 0:00:016m
0:00:0100:01
Downloading numpy-2.5.3-cp314-cp314-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (16.7 MB)
_____ 16.7/16.7 MB 39.4 MB/s 0:00:006m
0:00:01
Installing collected packages: numpy, opencv-python
_____ 2/2 [opencv-python]0m [opencv-python]
Successfully installed numpy-2.5.3 opencv-python-5.0.0.93
```

```
In [6]: import sys
        !{sys.executable} -m pip install matplotlib
```

```

Collecting matplotlib
  Downloading matplotlib-3.11.1-cp314-cp314-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (80 kB)
Collecting contourpy>=1.0.1 (from matplotlib)
  Downloading contourpy-1.3.3-cp314-cp314-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (5.5 kB)
Collecting cycler>=0.10 (from matplotlib)
  Downloading cycler-0.12.1-py3-none-any.whl.metadata (3.8 kB)
Collecting fonttools>=4.28.2 (from matplotlib)
  Downloading fonttools-4.64.0-cp314-cp314-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (123 kB)
Collecting kiwisolver>=1.3.1 (from matplotlib)
  Downloading kiwisolver-1.5.1-cp314-cp314-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (5.2 kB)
Requirement already satisfied: numpy>=1.25 in /usr/local/python/3.14.2/lib/python3.14/site-packages (from matplotlib) (2.5.3)
Requirement already satisfied: packaging>=20.0 in /home/codespace/.local/lib/python3.14/site-packages (from matplotlib) (26.3)
Collecting pillow>=9 (from matplotlib)
  Downloading pillow-12.3.0-cp314-cp314-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (9.1 kB)
Collecting pyparsing>=3 (from matplotlib)
  Downloading pyparsing-3.3.2-py3-none-any.whl.metadata (5.8 kB)
Requirement already satisfied: python-dateutil>=2.7 in /home/codespace/.local/lib/python3.14/site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /home/codespace/.local/lib/python3.14/site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Downloading matplotlib-3.11.1-cp314-cp314-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (11.1 MB)
_____ 11.1/11.1 MB 26.1 MB/s 0:00:00m
0:00:01
Downloading contourpy-1.3.3-cp314-cp314-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (363 kB)
Downloading cycler-0.12.1-py3-none-any.whl (8.3 kB)
Downloading fonttools-4.64.0-cp314-cp314-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (5.3 MB)
_____ 5.3/5.3 MB 37.3 MB/s 0:00:00
Downloading kiwisolver-1.5.1-cp314-cp314-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (1.5 MB)
_____ 1.5/1.5 MB 35.7 MB/s 0:00:00
Downloading pillow-12.3.0-cp314-cp314-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (6.9 MB)
_____ 6.9/6.9 MB 43.0 MB/s 0:00:00
Downloading pyparsing-3.3.2-py3-none-any.whl (122 kB)
Installing collected packages: pyparsing, pillow, kiwisolver, fonttools, cycler, contourpy, matplotlib
_____ 7/7 [matplotlib]7 [matplotlib]
Successfully installed contourpy-1.3.3 cycler-0.12.1 fonttools-4.64.0 kiwisolver-1.5.1 matplotlib-3.11.1 pillow-12.3.0 pyparsing-3.3.2

```

```

In [7]: import cv2
import numpy as np
import matplotlib.pyplot as plt

print("OpenCV version:", cv2.__version__)

```

OpenCV version: 5.0.0

```
In [9]: image = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)

if image is None:
    print("Image not found. Please keep image.jpg in the same folder as the
else:
    print("Image loaded successfully!")
    print("Image shape:", image.shape)
```

Image loaded successfully!
Image shape: (515, 388)

```
In [10]: plt.figure(figsize=(6, 5))
plt.imshow(image, cmap="gray")
plt.title("Original Grayscale Image")
plt.axis("off")
plt.show()
```

Original Grayscale Image



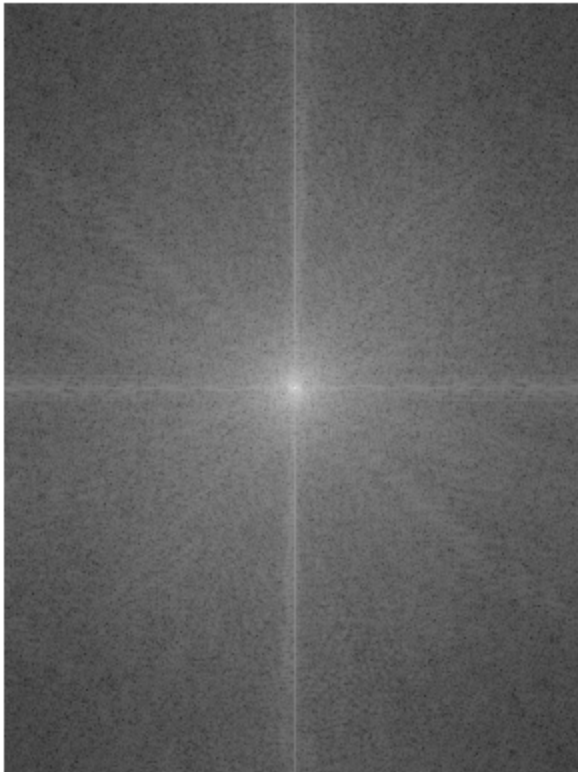
```
In [11]: dft = np.fft.fft2(image)
```

```
In [12]: dft_shift = np.fft.fftshift(dft)
```

```
In [13]: magnitude_spectrum = 20 * np.log(np.abs(dft_shift) + 1)

plt.figure(figsize=(6, 5))
plt.imshow(magnitude_spectrum, cmap="gray")
plt.title("Magnitude Spectrum")
plt.axis("off")
plt.show()
```

Magnitude Spectrum



```
In [14]: rows, cols = image.shape
        crow, ccol = rows // 2, cols // 2

        # Create a mask
        low_pass_mask = np.zeros((rows, cols), np.uint8)

        # Radius of the filter
        radius = 50

        # Create circular low-pass filter
        cv2.circle(low_pass_mask, (ccol, crow), radius, 1, -1)
```

```
Out[14]: array([[0, 0, 0, ..., 0, 0, 0],
               [0, 0, 0, ..., 0, 0, 0],
               [0, 0, 0, ..., 0, 0, 0],
               ...,
               [0, 0, 0, ..., 0, 0, 0],
               [0, 0, 0, ..., 0, 0, 0],
               [0, 0, 0, ..., 0, 0, 0]], shape=(515, 388), dtype=uint8)
```

```
In [15]: radius = 50
```

```
In [16]: low_pass_dft = dft_shift * low_pass_mask
```

Perform Inverse Fourier Transform for Low-Pass Image

```
In [17]: low_pass_ishift = np.fft.ifftshift(low_pass_dft)
```

```
In [18]: low_pass_image = np.fft.ifft2(low_pass_ishift)
```

```
In [19]: low_pass_image = np.abs(low_pass_image)
```

```
In [20]: low_pass_image = cv2.normalize(
    low_pass_image,
    None,
    0,
    255,
    cv2.NORM_MINMAX
).astype(np.uint8)
```

```
In [21]: high_pass_mask = np.ones((rows, cols), np.uint8)

cv2.circle(high_pass_mask, (ccol, crow), radius, 0, -1)
```

```
Out[21]: array([[1, 1, 1, ..., 1, 1, 1],
    [1, 1, 1, ..., 1, 1, 1],
    [1, 1, 1, ..., 1, 1, 1],
    ...,
    [1, 1, 1, ..., 1, 1, 1],
    [1, 1, 1, ..., 1, 1, 1],
    [1, 1, 1, ..., 1, 1, 1]], shape=(515, 388), dtype=uint8)
```

```
In [22]: high_pass_dft = dft_shift * high_pass_mask
```

```
In [23]: high_pass_ishift = np.fft.ifftshift(high_pass_dft)

high_pass_image = np.fft.ifft2(high_pass_ishift)

high_pass_image = np.abs(high_pass_image)
```

```
In [24]: high_pass_image = cv2.normalize(
    high_pass_image,
    None,
    0,
    255,
    cv2.NORM_MINMAX
).astype(np.uint8)
```

```
In [25]: plt.figure(figsize=(14, 10))

plt.subplot(2, 2, 1)
plt.imshow(image, cmap="gray")
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(magnitude_spectrum, cmap="gray")
plt.title("Magnitude Spectrum")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(low_pass_image, cmap="gray")
```

```
plt.title("Low-Pass Filtered Image")
plt.axis("off")

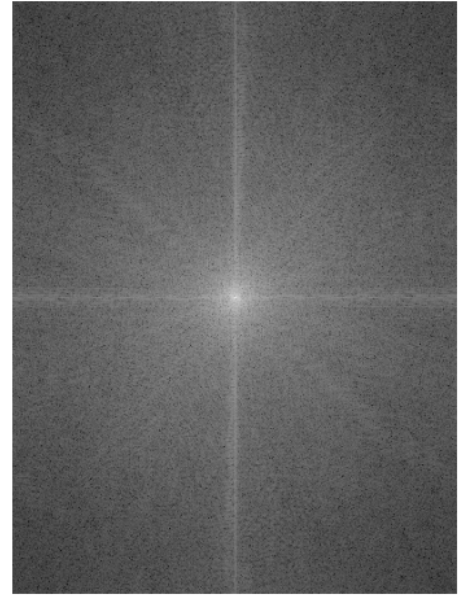
plt.subplot(2, 2, 4)
plt.imshow(high_pass_image, cmap="gray")
plt.title("High-Pass Filtered Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Original Image



Magnitude Spectrum



Low-Pass Filtered Image



High-Pass Filtered Image



After running the experiment, write these observations in your practical file:

Observation The Fourier Transform converts the spatial image into its frequency representation. The magnitude spectrum shows the distribution of frequency components in the image. Low-frequency components are concentrated around the

center of the shifted spectrum. The Low-Pass Filter removes high-frequency components and produces a smoother image. Fine details and sharp edges are reduced after Low-Pass filtering. The High-Pass Filter suppresses low-frequency components and emphasizes edges and fine details. Frequency domain filtering provides effective control over different frequency components of an image.